GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 14 OF 42 | CORE 90 CHAPTER TOPIC

Core 90 - Arrays and prefix sums

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Arrays and prefix sums | 3 questions | tap a title to jump to it

M 1. Longest Same-Color Grid Sequence — **1Point3Acres 2026-09-08**

M 2. Add two big integers from digit lists — **PracHub 2025-11-24**

M 3. Rotate Any m×n 2D Matrix — **1Point3Acres no date**

STUDY PATH: question 73 of 167 (Core 90)

Priority score 5.28 | Arrays and prefix sums | asked 42 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 1 of 3

Longest Same-Color Grid Sequence

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **ALL TIME 19.6** | **30 DAYS 0** | **PRIORITY 148/228**

ABOUT THIS QUESTION

Topic: Core 90 - Arrays and prefix sums

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix (related)

Frequency: Single report | Last asked: 2026-09-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

QUESTION

Given a grid of colors, return the longest consecutive same-color sequence along a horizontal, vertical, or diagonal line.

Input is a two-dimensional grid of colors such as R, G, and B. Return the length of the longest consecutive sequence of a single color horizontally, vertically, or diagonally.

EXAMPLES

```
R R R G
R R B B
G R R R
B G B R

Answer: 4
```

APPROACH

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

HOW TO THINK ABOUT IT

A straight line can run in four directions (right, down, down-right, down-left). For each cell and each direction keep the length of the same-colour run that ends there: it is the predecessor's run plus one if the colours match, otherwise 1. Because every run only looks one cell back in a fixed direction, one pass in reading order is enough.

Basis: LeetCode 562, Longest Line of Consecutive One in Matrix, generalised to any colour.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = 0
    for r, c in all_cells(grid):
        for dr, dc in DIRECTIONS:
            n = 1
            while same(grid, r, c,
                       r + n * dr, c + n * dc):
                n += 1
            best = max(best, n)
    return best
```

TIME	SPACE
O(m * n * max(m, n))	O(1)

WHY IT IS SLOW

Re-walks the same cells in every line, O(m * n * max(m, n)). Keeping the run length ending at each cell in each direction is O(m * n).

OPTIMAL SOLUTION · Four run lengths per cell

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The line may also bend (a connected region).

Then it is a different problem: the largest connected same-colour region, found with BFS/DFS or union-find in O(m * n). The run-length DP only handles straight lines because each run depends on exactly one predecessor.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Longest Same-Color Grid Sequence
#
# Given a grid of colors, return the longest consecutive
# same-color sequence along a horizontal, vertical, or
# diagonal line. Input is a two-dimensional grid of colors
# such as R, G, and B. Return the length of the longest
# consecutive sequence of a single color horizontally,
# vertically, or diagonally. Examples: R R R G R R B B G R
# R R B G B R Answer: 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. The line may also bend (a connected region).
#
# INTERVIEW NOTES
# A straight line can run in four directions (right, down,
# down-right, down-left). For each cell and each direction
# keep the length of the same-colour run that ends there:
# it is the predecessor's run plus one if the colours
# match, otherwise 1. Because every run only looks one
```

```
# cell back in a fixed direction, one pass in reading
# order is enough.
#
# BRUTE FORCE (say it first): Extend a line from every
# cell in every direction
# Start at each cell and walk in each of the four
# directions while the colour matches. Correct; it re-
# walks the same cells many times. Time O(m * n * max(m,
# n)). Space O(1).


# BRUTE FORCE (say it first): Extend a line from every
# cell in every direction | time O(m * n * max(m, n))
def longest_line_brute(g):
    best = 0
    for i in range(len(g)):
        for j in range(len(g[0])):
            for di, dj in (
                (0, 1),
                (1, 0),
                (1, 1),
                (1, -1),
            ):
                k, a, b = 0, i, j
                while (
                    0 <= a < len(g)
                    and 0 <= b < len(g[0])
                    and g[a][b] == g[i][j]
                ):
                    k += 1
                    a += di
                    b += dj
                best = max(best, k)
    return best


# BASE QUESTION (optimal): Four run lengths per cell
# Idea: Keep, for the previous row, the run length ending
# at each column in the vertical and both diagonal
# directions, and a running value for the horizontal run
# in the current row.
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n, best = len(g[0]), 0
    # run lengths ending in the previous row:
    # straight down, down-left, down-right
    up, dl, dr = [0] * n, [0] * n, [0] * n
    for i, row in enumerate(g):
        nu, nl, nr = [1] * n, [1] * n, [1] * n
        left = 0
        for j, color in enumerate(row):
            # horizontal run from the cell on the left
            same = j and row[j - 1] == color
            left = left + 1 if same else 1
            if i:
                # extend the run from the cell above
                if g[i - 1][j] == color:
                    nu[j] = up[j] + 1
                # from above-left (down-right diagonal)
                if j and g[i - 1][j - 1] == color:
                    nr[j] = dr[j - 1] + 1
                # from above-right (down-left diagonal)
                if j + 1 < n and g[i - 1][j + 1] == color:
                    nl[j] = dl[j + 1] + 1
            best = max(best, left, nu[j], nl[j], nr[j])
        up, dl, dr = nu, nl, nr
    return best


# FOLLOW-UP 1: The line may also bend (a connected
# region).
# Answer: Then it is a different problem: the largest
# connected same-colour region, found with BFS/DFS or
# union-find in O(m * n). The run-length DP only handles
# straight lines because each run depends on exactly one
# predecessor.
def largest_region(g):
    # follow-up: the line may bend (connected region)
    seen, best = set(), 0
    for r in range(len(g)):
        for c in range(len(g[0])):
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack, size = [(r, c)], 0
            while stack:
                x, y = stack.pop()
                size += 1
                for a, b in ((x + 1, y), (x - 1, y),
                             (x, y + 1), (x, y - 1)):
                    if a < 0 or b < 0:
                        continue
                    if a >= len(g) or b >= len(g[0]):
                        continue
                    if (a, b) not in seen \
                            and g[a][b] == g[r][c]:
                        seen.add((a, b))
                        stack.append((a, b))
            best = max(best, size)
    return best


if __name__ == "__main__":
    grid = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_same_color_line(grid) == 4
    assert longest_same_color_line([]) == 0
    # one column and alternating colours
    assert longest_same_color_line(["R", "R", "R"]) == 3
    assert longest_same_color_line(["RGR", "GRG"]) == 2
    # a bent region is longer than any straight line
    bent = ["RRR", "GGR"]
    assert longest_same_color_line(bent) == 3
    assert largest_region(["RR", "GR"]) == 3
```

NOTES FROM CANDIDATES

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.

Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.

Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

PREPARATION

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

STUDY PATH: question 74 of 167 (Core 90)

Priority score 4.15 | Arrays and prefix sums | asked 330 days ago

MEDIUM | IN APP LC 2 | PracHub | Technical Screen | 2 of 3

★ TOP 75: rank 1 in the Medium and Hard priority list

★ Add two big integers from digit lists

POSTED 2025-11-24 | 330 DAYS AGO | OLDER

STUDY: TYPE | **ALL TIME 78.7** | **30 DAYS 69.3** | **PRIORITY 1/228**

ABOUT THIS QUESTION

Topic: Core 90 - Arrays and prefix sums

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #2 Add Two Numbers

LeetCode match: LC 2 Add Two Numbers

Posted on PracHub: 2025-11-24 (updated 2026-06-26)

Original:
https://prachub.com/coding-questions/add-two-big-integers-from-digit-lists

QUESTION

You are given two non-empty singly linked lists (or arrays) representing two non-negative integers. The digits are stored in **reverse order**, and each node contains a single digit from 0 to 9. Your task is to add the two numbers and return the sum as a new linked list, also in reverse order. You may assume:

- Each list has at least one node.
- The integers do not contain leading zeros, except the number 0 itself.

Example

Input:

- List 1: 2 -> 4 -> 3 (represents the integer 342)
- List 2: 5 -> 6 -> 4 (represents the integer 465)

Output:

- 7 -> 0 -> 8 (represents the integer 807, since 342 + 465 = 807)

Another example

Input:

- List 1: 9 -> 9 -> 9 (represents 999)
- List 2: 1 (represents 1)

Output:

- 0 -> 0 -> 0 -> 1 (represents 1000)

Constraints

- Let n be the length of the first list and m be the length of the second list.
- 1 <= n, m <= 10^5.
- Each node value is an integer in the range [0, 9].

Task: Describe an algorithm to perform this addition and construct the resulting linked list in O(n + m) time and O(1) additional space (excluding the space for the output list).

QUESTION

You are given two non-empty arrays of digits representing two non-negative integers. The digits are stored in **reverse order** (least significant digit first), and each element is a single digit from 0 to 9. Add the two numbers and return the sum as a new array of digits, also in reverse order.

Assumptions

- Each array has at least one element.
- The integers do not contain leading zeros, except the number 0 itself.

Example 1

```
Input: l1 = [2, 4, 3], l2 = [5, 6, 4] (342 + 465)
Output: [7, 0, 8] (807)
```

Example 2

```
Input: l1 = [9, 9, 9], l2 = [1] (999 + 1)
Output: [0, 0, 0, 1] (1000)
```

Implement **`add_two_numbers(l1, l2)`** (camelCase **`addTwoNumbers`** in Java/C++/JS) returning the digit array of the sum in reverse order. Aim for O(n + m) time and O(1) extra space beyond the output.

Constraints

1 <= n, m <= 10^5 where n = len(l1), m = len(l2)Each element is an integer in the range [0, 9]Digits are stored in reverse order (least significant digit first)No leading zeros except for the number 0 itself

EXAMPLES

```
Input: ([2, 4, 3], [5, 6, 4])
Expected Output: [7, 0, 8]
Explanation: 342 + 465 = 807, stored reversed as [7,
    0, 8].
Input: ([9, 9, 9], [1])
Expected Output: [0, 0, 0, 1]
```

Explanation: 999 + 1 = 1000; carry propagates through all three digits and creates a new leading digit.

Hints

Walk both arrays from index 0 (the least significant digit) toward the end, just like grade-school column addition.Maintain a single carry variable. At each position, sum the available digits plus the carry; append (sum % 10) and set carry = sum // 10.When one array runs out, keep going with just the carry and the remaining longer array. Don't forget a final carry can add one extra digit (e.g. 999 + 1 -> [0,0,0,1]).

APPROACH

Walk both lists with a carry, build the result list. O(max(m, n)).

HOW TO THINK ABOUT IT

Addition by hand: start at the least significant end, add the two digits and the carry, write digit % 10, carry digit // 10. The only edge cases are different lengths and a final carry. Never convert to a machine integer in an interview about big numbers, since the point is that the numbers do not fit.

Basis: LeetCode 989 / 415 (Add to Array-Form of Integer, Add Strings). Assumed input: lists of decimal digits, most significant digit first.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def add_lists(a, b):
    x = digits_to_int(a)  # reverse-order digits
    y = digits_to_int(b)
    return int_to_digits(x + y)
```

TIME	SPACE
O(n^2) for the string-to-integer conversions	O(n)

WHY IT IS SLOW

It relies on arbitrary-size integers and the conversions are quadratic for very long inputs. Adding digit by digit from the right with a carry is O(max length).

OPTIMAL SOLUTION · Digit by digit with a carry

TIME	SPACE
O(max(m, n))	O(max(m, n))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The digits are stored least significant first in a linked list.

Then the head is already the low end, so walk both lists together with the same carry loop and build the result list as you go. No reversal needed (LeetCode 2).

FOLLOW-UP 2 Multiply instead of add.

Schoolbook multiplication: the product of digits a[i] and b[j] lands in position i + j + 1, with the carry spilling to i + j. O(m * n); Karatsuba or FFT for very long numbers.

FOLLOW-UP 3 Subtract, or numbers in another base.

Subtraction borrows instead of carrying and needs the larger number first (compare lengths, then digits). Another base replaces 10 with the base in % and //.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Add two big integers from digit lists
#
# You are given two non-empty singly linked lists (or
# arrays) representing two non-negative integers. The
# digits are stored in **reverse order**, and each node
# contains a single digit from 0 to 9. Your task is to add
# the two numbers and return the sum as a new linked list,
# also in reverse order. You may assume: - Each list has
# at least one node. - The integers do not contain leading
# zeros, except the number 0 itself. ### Example Input: -
# List 1: 2 -> 4 -> 3 (represents the integer 342) - List
2:
# 5 -> 6 -> 4 (represents the integer 465) Output: - 7 ->
0 ->
# 8 (represents the integer 807, since 342 + 465 = 807)
# ### Another example Input: - List 1: 9 -> 9 -> 9
# (represents 999) - List 2: 1 (represents 1) Output: - 0
# -> 0 -> 0 -> 1 (represents 1000) ### Constraints - Let n
 be
# the length of the first list and m be the length of the
# second list. - 1 <= n, m <= 10^5.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The digits are stored least significant first in a
# linked list.
# 2. Multiply instead of add.
# 3. Subtract, or numbers in another base.
#
# INTERVIEW NOTES
# Addition by hand: start at the least significant end,
# add the two digits and the carry, write digit % 10,
# carry digit // 10. The only edge cases are different
# lengths and a final carry. Never convert to a machine
# integer in an interview about big numbers, since the
# point is that the numbers do not fit.
#
# BRUTE FORCE (say it first): Convert both lists to
# integers and add
# Join the digits, int() them, add, and split the result
# into digits. It is a valid shortcut in Python (big
# integers), but it hides the algorithm, depends on the
# language having big integers, and the conversions are
# quadratic for very long inputs. Time O(n^2) for the
# string-to-integer conversions. Space O(n).


# BRUTE FORCE (say it first): Convert both lists to
# integers and add | time O(n^2) for the string-to-integer
```

```
# conversions
def add_digits_int(a, b):
    return [
        int(ch)
        for ch in str(
            int("".join(map(str, a)))
            + int("".join(map(str, b)))
        )
    ]


# HELPERS for follow-up 1
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
```

```
# BASE QUESTION (optimal): Digit by digit with a carry
# Idea: Walk both lists from the right with indices i and
# j; at each step add a[i] (or 0 once exhausted), b[j] (or
# 0) and the carry, append sum % 10, and carry = sum //
# 10.
def add_digits(a, b):
    # arrays with the most significant digit FIRST
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry
        total += a[i] if i >= 0 else 0
        total += b[j] if j >= 0 else 0
        out.append(total % 10)
        carry = total // 10
        i, j = i - 1, j - 1
    return out[::-1] or [0]


# FOLLOW-UP 1: The digits are stored least significant
# first in a linked list.
# Answer: Then the head is already the low end, so walk
# both lists together with the same carry loop and build
# the result list as you go. No reversal needed (LeetCode
# 2).
def add_lists(a, b):
    # digits least significant first: add with a carry,
    # one pass
    dummy = tail = ListNode()
```

```
    carry = 0
    while a or b or carry:
        total = carry
        total += a.val if a else 0
        total += b.val if b else 0
        tail.next = ListNode(total % 10)
        tail, carry = tail.next, total // 10
        a = a.next if a else None
        b = b.next if b else None
    return dummy.next


# FOLLOW-UP 2: Multiply instead of add.
# Answer: Schoolbook multiplication: the product of digits
# a[i] and b[j] lands in position i + j + 1, with the
# carry spilling to i + j. O(m * n); Karatsuba or FFT for
# very long numbers.
def multiply_digits(a, b):
    # schoolbook: digit a[i] * b[j] lands at position i +
    # j + 1
    res = [0] * (len(a) + len(b))
    for i in range(len(a) - 1, -1, -1):
        for j in range(len(b) - 1, -1, -1):
            total = a[i] * b[j] + res[i + j + 1]
            res[i + j + 1] = total % 10
            res[i + j] += total // 10
    while len(res) > 1 and res[0] == 0:
        res.pop(0)
    return res


# FOLLOW-UP 3: Subtract, or numbers in another base.
# Answer: Subtraction borrows instead of carrying and
# needs the larger number first (compare lengths, then
# digits). Another base replaces 10 with the base in % and
# //.
def subtract_digits(a, b):
    # a - b where a >= b (most significant first): borrow
    i, j, borrow, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0:
        d = a[i] - borrow - (b[j] if j >= 0 else 0)
        borrow = 1 if d < 0 else 0
        out.append(d + 10 * borrow)
        i, j = i - 1, j - 1
```

```
    while len(out) > 1 and out[-1] == 0:
```

```
        out.pop()
    return out[::-1]


# HELPERS for the checks below
def make(digits):
    # digits least significant first -> linked list
    head = tail = ListNode(digits[0])
    for d in digits[1:]:
        tail.next = ListNode(d)
        tail = tail.next
    return head


def to_digits(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out


if __name__ == "__main__":
    # 342 + 465 = 807, digits reversed
    got = add_lists(make([2, 4, 3]), make([5, 6, 4]))
    assert to_digits(got) == [7, 0, 8]
    # a final carry adds a digit: 99 + 1 = 100
    carry = add_lists(make([9, 9]), make([1]))
    assert to_digits(carry) == [0, 0, 1]
    assert add_digits([9, 9, 9], [1]) == [1, 0, 0, 0]
    assert add_digits([0], [0]) == [0]
    product = multiply_digits([1, 2, 3], [4, 5])
    assert product == [5, 5, 3, 5]
    assert multiply_digits([0], [9, 9]) == [0]
    assert subtract_digits([1, 0, 0], [1]) == [9, 9]
    assert subtract_digits([5, 0], [5, 0]) == [0]
```

FUNCTION AND TEST CASES

Sample call

```
add_two_numbers([2, 4, 3], [5, 6, 4])
```

Test cases

Input	Expected
([2, 4, 3], [5, 6, 4])	[7, 0, 8]
([9, 9, 9], [1])	[0, 0, 0, 1]

STUDY PATH: question 75 of 167 (Core 90)

Priority score 3.75 | Arrays and prefix sums | no date | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 3 of 3

Rotate Any m×n 2D Matrix

STUDY: TYPE | **ALL TIME 56.3** | **30 DAYS 0** | **PRIORITY 57/228**

ABOUT THIS QUESTION

Topic: Core 90 - Arrays and prefix sums

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #48 Rotate Image

LeetCode match: LC 48 Rotate Image (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/rotate-matrix-rectangle

QUESTION

EXAMPLES

```
[[1,2,3],[4,5,6]] (2×3) rotated 90° CW ->
    [[4,1],[5,2],[6,3]] (3×2).
```

APPROACH

Write square in-place rotation from memory in under 8 min. Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which direction and angle, whether in place is required, and the dimensions. A non-square matrix cannot be rotated in place in the same buffer, since its shape changes.

BRUTE FORCE TO SAY OUT LOUD

Create a new matrix and copy each cell to its rotated position with an index formula. O(mn) space.

OPTIMAL IDEA

Clockwise 90 degrees: reverse the rows and transpose, [list(r) for r in zip(*matrix[::-1])]. Counter-clockwise: transpose then reverse the rows. 180 degrees: reverse the rows and each row. For a square matrix rotate in place by cycling four cells per layer.

DRY-RUN THESE CASES

The 2 x 3 example giving [[4,1],[5,2],[6,3]], 1 x 1, 1 x n, an empty matrix, four rotations returning the original.

LIKELY FOLLOW-UPS

In place square rotation, rotation by k times 90 degrees, or non-square in place with a flat array and cycles.

PITFALLS

Mixing up the index formula direction. Trying to rotate a rectangle in place in a 2D list.

HOW TO THINK ABOUT IT

A 90 degree clockwise rotation sends the cell at row i, column j to row j, column (m - 1 - i) of the new n x m matrix. That formula is the whole problem; 'reverse the rows then transpose' is just a short way to write it. A rectangle cannot be rotated in place because its shape changes.

Basis: Generalizes LeetCode 48, Rotate Image, from an n x n matrix to any m x n matrix.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def rotate(old):
    m, n = len(old), len(old[0])
    new = [[0] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = old[i][j]
    return new
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

This is already optimal in time, O(m * n); its cost is the second matrix, O(m * n) extra space. A rectangle cannot be rotated in place (its shape changes), so it is the expected answer; the zip(reversed rows) one-liner is the same idea.

OPTIMAL SOLUTION · Rotate 90 degrees clockwise by reverse + transpose

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can a non-square matrix be rotated in place?

Not in a rectangular buffer, because the dimensions swap. You can do it in place only if the matrix is stored as a flat array and you permute the elements along cycles of the index map (i, j) to (j, m - 1 - i), which is O(mn) time but needs cycle bookkeeping. In an interview, allocate the new matrix and say why.

FOLLOW-UP 2 How would you rotate by k * 90 degrees?

Reduce k modulo 4 first. Then apply the single-turn function that many times, or write the four direct index maps so you only build the output once.

FOLLOW-UP 3 Rotate by k quarter turns.

Reduce k modulo 4 first; four turns are the identity. Apply the single-turn function that many times (at most three), or write the three index formulas directly to build the answer once.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Rotate Any m×n 2D Matrix
#
# Examples: [[1,2,3],[4,5,6]] (2×3) rotated 90° CW ->
# [[4,1],[5,2],[6,3]] (3×2).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can a non-square matrix be rotated in place?
# 2. How would you rotate by k * 90 degrees?
# 3. Rotate by k quarter turns.
#
# INTERVIEW NOTES
# A 90 degree clockwise rotation sends the cell at row i,
# column j to row j, column (m - 1 - i) of the new n x m
# matrix. That formula is the whole problem; 'reverse the
# rows then transpose' is just a short way to write it. A
# rectangle cannot be rotated in place because its shape
# changes.
#
# BRUTE FORCE (say it first): Allocate the new matrix and
# apply the index formula
# Create an n x m matrix and copy each cell old[i][j] to
# new[j][m - 1 - i]. Correct and the clearest statement of
# the rotation; the zip trick below is the same thing in
# one line. Time O(m * n). Space O(m * n).


# BRUTE FORCE (say it first): Allocate the new matrix and
# apply the index formula | time O(m * n)
def rotate_by_formula(matrix):
    m = len(matrix)
    if m == 0:
        return []
    n = len(matrix[0])
    new = [[None] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = matrix[i][j]
    return new


# BASE QUESTION (optimal): Rotate 90 degrees clockwise by
# reverse + transpose
# Idea: Rotating clockwise puts the last row first, so
# reverse the rows and transpose (zip with *). A non-
# square matrix changes shape from m x n to n x m, which
```

```
# is why it cannot be done in place the way LeetCode 48
# is.
def rotate_cw(matrix):
    # reverse the rows, then swap rows and columns
    return [list(row) for row in zip(*matrix[::-1])]


# COUNTER-CLOCKWISE
def rotate_ccw(matrix):
    # swap rows and columns, then reverse the rows
    return [list(row) for row in zip(*matrix)][::-1]


# INDEX RULE version
def rotate_by_index(matrix):
    # the same result from the index rule, built once:
    # new[j][m - 1 - i] = old[i][j]
    m, n = len(matrix), len(matrix[0]) if matrix else 0
    new = [[None] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = matrix[i][j]
    return new


# FOLLOW-UP 2: How would you rotate by k * 90 degrees?
# Answer: Reduce k modulo 4 first. Then apply the single-
# turn function that many times, or write the four direct
# index maps so you only build the output once.
# FOLLOW-UP 3: Rotate by k quarter turns.
# Answer: Reduce k modulo 4 first; four turns are the
# identity. Apply the single-turn function that many times
# (at most three), or write the three index formulas
# directly to build the answer once.
def rotate_k(matrix, k):
    # four quarter turns are the identity: use k mod 4
    for _ in range(k % 4):
        matrix = rotate_cw(matrix)
    return matrix


if __name__ == "__main__":
    a = [[1, 2, 3], [4, 5, 6]]
    # the 2 x 3 example becomes 3 x 2
    assert rotate_cw(a) == [[4, 1], [5, 2], [6, 3]]
```

```
    assert rotate_by_index(a) == rotate_cw(a)
    assert rotate_ccw(a) == [[3, 6], [2, 5], [1, 4]]
    # a clockwise then an anticlockwise turn cancel out
    assert rotate_ccw(rotate_cw(a)) == a
    assert rotate_k(a, 4) == a
    assert rotate_k(a, 2) == [[6, 5, 4], [3, 2, 1]]
    assert rotate_k(a, 5) == rotate_cw(a)
    assert rotate_cw([[7]]) == [[7]]
    assert rotate_cw([]) == []
```

NOTES FROM CANDIDATES

The in-place rectangle rotation is essentially cycle-decomposition on the index permutation — explain it but don't burn time implementing.
Reported follow-up: rotate inside a window of a larger matrix while keeping the rest untouched.

PREPARATION

Write square in-place rotation from memory in under 8 min. Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.